

FAMILIARISATION ABOUT PYTHON PROGRAMMING

AIM: To familiarise about Python Programming

THEORY:

Python is an easy to learn, powerful programming language. It has efficient high-level data structures and a simple but effective approach to object-oriented programming. It is a general purpose, interpreted, interactive, object oriented, and high-level programming language. Python is designed to be highly readable. It uses English keywords frequently where as other languages use punctuation, and it has fewer syntactical constructions than other languages.

Control Structures

A program's control flow is the order in which the program's code executes. The control flow of a python program is regulated by conditional statements, loops and function calls. Python has three types of control structures:

Sequential-default mode

Selection-used for decisions and branching

Repetition-used for looping;ie.,repeating a piece of code multiple times

1. Sequential Statements

Sequential statements are a set of statements whose execution process happens in a sequence. The problem with sequential statements is that if the logic has broken in any one of the lines, then the complete source code execution will break.

2. Selection/Decision control statements

In python, the selection statements are also known as Decision control statements. The selection statement allows a program to test several conditions and execute instructions based on which condition is true.

Some decision control statements are :

- Simple if
- if-else
- Nested if
- if-elif-else

Simple if

Syntax:

```
if test_condition:  
    Statements
```

if-else

Syntax:

```
if test_condition:  
    Statements1  
else:  
    Statements2
```

Nested if

Syntax:

```
if test_condition:  
    if test_condition:
```

```

        if statement
    else:
        else 1 statement
else:
    else statements

```

if-elif-else

Syntax:

```

    if test_condition:
        ifstatement
    elif test_condition:
        elif statement1
    elif test_condition:
        elif statement2
else:
    else statements

```

3. Repetition statements

Repetition is used to repeat a group(block) of programming instructions. In python, we generally have two loops, repetitive statements:

- while loop
- for loop

while loop

In python, while loops are used to execute a block of statements repeatedly until a given condition is satisfied. Then, the expression is checked again and, if it is still true, the body is executed again. This continues until the expression becomes false.

Syntax:

```

while(test_condition):
    Statements

```

for loop

A for loop is used to iterate over a sequence that is either a list, tuple, dictionary, or a set. We can execute a set of statements once for each item in a list, tuple, or dictionary.

Syntax:

```

for variable in range(start value, end value):
    Statements

```

Different Datatypes in Python

In Python, **data types** define the kind of values a variable can hold and determine the operations that can be performed on them. Since Python is **dynamically typed**, you don't need to declare the type explicitly — it's inferred from the assigned value.

Main built-in categories include:

- **Numeric:** `int`, `float`, `complex`
- **Sequence:** `str`, `list`, `tuple`, `range`
- **Mapping:** `dict`
- **Set:** `set`, `frozenset`
- **Boolean:** `bool`
- **Binary:** `bytes`, `bytearray`, `memoryview`
- **NoneType:** `None`

1. Numeric Types

- **int:** Whole numbers without decimals.
- **float:** Numbers with decimals or in scientific notation.
- **complex:** Numbers with real and imaginary parts (`a + bj`).

```
a = 5 # int
b = 3.14 # float
c = 2 + 3j # complex
```

2. Sequence Types

- **str**: Immutable text data.
- **list**: Ordered, mutable collection.
- **tuple**: Ordered, immutable collection.

```
s = "Hello" # str
lst = [1, "apple", 3.5] # list
tup = (10, 20, 30) # tuple
```

3. Mapping Type

- **dict**: Key-value pairs, keys must be immutable.

```
person = {"name": "Alice", "age": 25}
```

4. Set Types

- **set**: Unordered, mutable, unique elements.
- **frozenset**: Immutable version of a set.

```
colors = {"red", "blue"}
immutable_colors = frozenset(["red", "blue"])
```

5. Boolean Type

- **bool**: Logical values `True` or `False`.

```
is_active = True
```

6. Binary Types

- **bytes**: Immutable binary data.
- **bytearray**: Mutable binary data.
- **memoryview**: Memory-efficient view of binary data.

```
b = b"data"
ba = bytearray(b"data")
mv = memoryview(ba)
```

7. NoneType

- **None**: Represents absence of a value.

```
result = None
```

Type Checking & Conversion Use `type()` or `isinstance()` to check types, and constructors like `int()`, `float()`, `str()` for explicit conversion.

```
x = "123"
print(type(x)) # <class 'str'>
y = int(x) # convert to int
```

RESULT:

Familiarised about Python Programming

Exp. No.2

PERSONAL INFORMATION

AIM: Program to print Personal Information showing different data types.

PROGRAM

```
name = input("Enter your Name: ")      # String
age = int(input("Enter your Age: "))    # Integer
height = float(input("Enter your Height: ")) # Float
student = bool(int(input("Are you a student? (1 for Yes / 0 for No): "))) # Boolean

print("\n--- Personal Information ---")
print("Name:", name)
print("Age:", age)
print("Height:", height)
print("Student:", student)
```

RESULT

Program executed successfully and obtained the output

OUTPUT

```
Enter yourName: Anoop
Enter your Age: 20
Enter your Height: 5.7
Are you a student? (1 for Yes / 0 for No): 1
```

```
--- Personal Information ---
```

```
Name: Anoop
```

```
Age: 20
```

```
Height: 5.7
```

```
Student: True
```

Exp. No. 3

LIST MANAGEMENT

AIM : Program for a list of fruits (to append, remove, insert, reverse, sort).

PROGRAM

```
# List of fruits
fruits = ["Apple", "Mango", "Orange"]
print("Initial List:", fruits)

# Append
fruits.append("Banana")
print("After append:", fruits)

# Remove
fruits.remove("Mango")
print("After remove:", fruits)

# Insert
fruits.insert(1, "Grapes")
print("After insert:", fruits)

# Reverse
fruits.reverse()
print("After reverse:", fruits)

# Sort
fruits.sort()
print("After sort:", fruits)
```

RESULT

Program executed successfully and obtained the output

OUTPUT

```
Initial List: ['Apple', 'Mango', 'Orange']
After append: ['Apple', 'Mango', 'Orange', 'Banana']
After remove: ['Apple', 'Orange', 'Banana']
After insert: ['Apple', 'Grapes', 'Orange', 'Banana']
After reverse: ['Banana', 'Orange', 'Grapes', 'Apple']
After sort: ['Apple', 'Banana', 'Grapes', 'Orange']
```

Exp. No. 4

DICTIONARY MANAGEMENT

AIM : Program using a dictionary to store marks of different subjects and Create a function grade(mark) to determine the grade based on the following conditions:

- 90 and above → A+
- 80 - 89 → A
- 70 - 79 → B+
- 60 - 69 → B
- 50 - 59 → C
- Below 50 → Fail

Display the subject name, mark, and grade in a properly aligned table format.

PROGRAM

```
subjects={
    "Maths":85,
    "Physics":72,
    "Chemisty":64,
    "English":90,
    "Computer":30
}

def grade(mark):
    if mark>=90:
        return "A+"
    if mark>=80:
        return "A"
    if mark>=70:
        return "B+"
    if mark>=60:
        return "B"
    if mark>=50:
        return "C"
    else:
        return "Fail"

print("Subject mark Grade")
print("-----")
for sub,mark in subjects.items():
    print(sub,"",mark,"",grade(mark))
```

```
print(f{'Subject':<12}{ 'Mark':<8}{ 'Grade':<6})  
print("-"*26)  
  
for sub, mark in subjects.items():  
    print(f"{sub:<12}{mark:<8}{grade(mark):<6}")
```

RESULT

Program executed successfully and obtained the output

OUTPUT

Subject	Mark	Grade
Maths	85	A
Physics	72	B+
Chemistry	64	B
English	90	A+
Computer	30	Fail

Exp. No. 5

LARGEST FROM THREE NUMBERS

AIM: Program to print largest from three numbers using *nested...if, if...elif..else.*

PROGRAM

```
#largest among three nos.  
a=float(input("Enter the First No."))  
b=float(input("Enter the Second No."))  
c=float(input("Enter the Third No."))  
if a>b:  
    if a>c:  
        print (a,'is largest')  
    else:  
        print (c,'is largest')  
elif b>c:  
    print (b,'is largest')  
else:  
    print (c,'is largest')
```

RESULT:

Program is executed successfully and output is obtained.

OUTPUT

```
Enter the First No. 15  
Enter the Second No. 25  
Enter the Third No. 20  
  
25 is largest
```


SIMPLE CALCULATOR USING MATCH....CASE

AIM: Program to implement simple calculator using 'match...case'

PROGRAM

```
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))

print("\ t1.ADDITION")
print("\ t2.SUBTRACTION")
print("\ t3.MILTIPLICATION")
print("\ t4.DIVISION")
choice = input("Choice: ")

match choice:
    case "1":
        print("Sum = ", a + b)
    case "2":
        print("Difference = ", a - b)
    case "3":
        print("Product = ", a * b)
    case "4":
        if b != 0:
            print("Quotient = ", a / b)
        else:
            print("Division not possible")
    case _:
        print("Invalid choice")
```

RESULT:

Program is executed successfully and output is obtained.

OUTPUT

Enter first number: 25

Enter second number: 2

1.ADDITION

2.SUBTRACTION

3.MILTIPLICATION

4.DIVISION

Choice: 4

Quotient = 12.5